Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?
  Un arreglo vector de n posiciones. Cada posición guarda una tarjeta y el orden de la fila es el orden de los índices, igual que cuando nos paramos uno junto al otro.

- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?
  El ciclo de afuera i son las pasadas, y cada vez que da una vuelta equivale a recorrer toda la fila una vez. El ciclo de adentro j son las comparaciones de pares vecinos dentro de esa pasada.

- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?
  Con una variable auxiliar. Primero guardamos en ella la tarjeta de la izquierda, luego pasamos la de la derecha a la izquierda y al final ponemos la guardada a la derecha. Si lo hiciéramos directo, al copiar una encima de la otra se perdería un valor. Es como cuando dos personas se cambian de lugar y una tiene que esperar un momento aparte.

- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?
  Al empezar cada pasada se pone en falso, y si en algún momento se hace un intercambio se pone en verdadero. Si la pasada termina y sigue en falso, significa que todos los pares estaban en orden, entonces ya no hay nada que acomodar y se sale del ciclo sin hacer las pasadas que faltaban. En nuestro ejemplo eso pasó en la pasada 4.

- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.
  Cada pasada deja la tarjeta más grande al final de la fila, así que esas ya no hay que volver a revisarlas. En la pasada i se hacen **n − i** comparaciones. Con 5 tarjetas serían 4, 3, 2, 1 y 0.

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
  Sí. Al meter los mismos números de la fila (5, 3, 8, 1, 4), el programa dio las mismas 4 pasadas, 10 comparaciones y 6 intercambios de nuestra tabla, y la fila quedó igual al final de cada pasada.

- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
  La lógica fue la misma, solo cambió cómo se escribe. Los arreglos en Java empiezan en 0 y no en 1, así que ajustamos los ciclos. También cambiamos la flecha <- por =, declaramos el tipo de cada variable, y el "salir del ciclo" se volvió un break.

- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
  Hace una sola pasada de 4 comparaciones y se detiene, porque la bandera huboIntercambio se queda en falso al no mover nada. Sin esa bandera habría hecho las 4 pasadas completas, o sea 10 comparaciones para nada.

- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$).
  No. Las comparaciones crecen como $n(n-1)/2$, o sea O(n2) con 100 elementos son 4950 y con 1000 ya son casi 500 mil Si la lista se duplica, el trabajo se cuadruplica. Nos sirvió para entender cómo ordena paso a paso, pero para listas grandes es mejor usar algoritmos como merge sort o quicksort.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			